

파이썬 함수·문법 정리

수업 PPT 6개(02-1 ~ 04-4)에 나온 함수, 메서드, 문법을 장별로 모았습니다. 예시 코드는 슬라이드 예제를 바탕으로 했습니다.

02 기초 문법

연산자 · round · print · range
· for · f문자열 · try-except ·
lambda · def · input · int/
float · if-else

03 텍스트 파일 가공

os 모듈 · open/close · 파일 모드 · read/write/
seek · with · encoding ·
re(match · search · findall · split · sub · compile)
· replace · set

04 CSV 데이터

csv.reader · csv.writer ·
writerows · opencsv() ·
writescsv() · switch() ·
usecsv 모듈 · 문자→숫자 전환

02 기초 문법으로 프로그램 만들기

▶ 기본 연산자

연산자	의미	예시	결과
<code>+</code> <code>-</code> <code>*</code>	더하기 · 빼기 · 곱하기	<code>5 * 2</code>	10
<code>**</code>	제곱	<code>2 ** 4</code>	16
<code>/</code>	나누기 (소수점까지)	<code>10 / 2</code>	5.0
<code>//</code>	나눗셈의 몫	<code>11 // 2</code>	5
<code>%</code>	나눗셈의 나머지	<code>8 % 5</code>	3

연산자 우선순위

나눗셈·곱셈이 덧셈보다 먼저 계산된다. `50 + 45 + 33 + 39 + 29 + 30 / 6` → 201.0 이므로 평균을 구하려면 `(50 + 45 + 33 + 39 + 29 + 30) / 6` 처럼 괄호를 써야 한다.

▶ 자주 쓰는 내장 함수

함수	형식	설명
<code>print()</code>	<code>print(출력 내용)</code>	결과를 화면에 출력
<code>round()</code>	<code>round(숫자, 표시할 소수점 자릿수)</code>	지정한 자리까지 반올림. <code>round(17/7, 1)</code> → 2.4
<code>range()</code>	<code>range(시작할 정수, 끝나는 정수 + 1)</code>	정수열을 만들. <code>range(2, 20)</code> → 2부터 19까지
<code>input()</code>	<code>input('안내 문구')</code>	사용자에게 값을 입력받음. 항상 문자(str)로 저장
<code>int()</code> / <code>float()</code>	<code>int(a)</code> / <code>float(a)</code>	정수형 / 실수형으로 변환
<code>type()</code>	<code>type(객체)</code>	자료형 확인. 예) <code><class 'int'></code>

```
>>> a = input('숫자를 하나 입력해 보세요: ')
숫자를 하나 입력해 보세요: 23
>>> a
'23' # 문자로 저장됨 → 계산하려면 int(a) / float(a)
>>> a = int(a); type(a)
<class 'int'>
```

▶ for 문과 19단 곱셈표

문법	형식 / 설명
for	for 변수 in 리스트(또는 튜플, 문자열): 다음 줄에 들어서서 수행할 문장 . 원소를 하나씩 변수에 넣어 반복
f 문자열	f'{변수} 문자' — 중괄호 안에 변수를 바로 넣어 출력
% 포매팅	print('%d를 포함한 출력 내용' % 넣을 숫자)

```
>>> for item in range(2, 20):
    for each in range(2, 20):
        print(f'{item} X {each} = {item * each}')
2 X 2 = 4 ... 19 X 19 = 361
```

▶ 예외 처리: try - except - finally

블록	역할
try	예외가 발생할 가능성이 있는 코드
except	예외가 발생했을 때 실행할 코드 (except 오류이름: 으로 특정 오류만 처리 가능)
finally	예외 발생 여부와 상관없이 항상 실행

```
>>> try:
    print(1 / 0)
except ZeroDivisionError:
    print("Division by zero error")
finally:
    print("This will always be printed")
```

▶ 오류 종류 10가지

오류	발생 상황	예
NameError	이름을 정의하지 않음	<code>print(foo)</code>
TypeError	잘못된 유형의 값 사용	<code>print(1 + "hello")</code>
ValueError	잘못된 값 사용	<code>int("hello")</code>
IndexError	인덱스가 범위를 벗어남	<code>[1, 2, 3][3]</code>
KeyError	딕셔너리에 없는 키에 접근	<code>my_dict["key"]</code>
AttributeError	개체에 없는 속성에 접근	<code>my_object.attribute</code>
SyntaxError	잘못된 문법으로 코드 작성	콜론(:) 누락 등
IndentationError	들여쓰기가 올바르지 않음	if 다음 줄 들여쓰기 누락

ImportError	모듈을 임포트할 수 없음	<code>import my_module</code>
ZeroDivisionError	값을 0으로 나눔	<code>1 / 0</code>

▶ 함수 만들기: lambda와 def

lambda — 한 줄로 함수 만들기 (람다 함수, 람다 표현식)

```
>>> add = lambda a, b : a + b
>>> add(2, 3)
5
>>> short = lambda x : x[:10]
>>> exchange = lambda won : won * 0.00086
>>> exchange(1000000)
860.0
```

def — 함수 정의 기본 명령어

```
def 함수이름(매개변수):    # 콜론, 들여쓰기 주의
    수행할 문장
    return 결과값

>>> def calculator(a, b):
        return a + b, a - b, a * b, a / b
>>> calculator(12, 3)
(15, 9, 36, 4.0)    # 여러 값 → tuple
```

주의

lambda를 남발하면 가독성이 떨어지므로 def 문을 권장. 함수에서 `print()` 만 하고 `return` 하지 않으면 반환값이 None이라 다른 연산에 쓸 수 없다.

▶ if - else 문

```
if 조건식:
    실행할 동작
else:
    실행할 동작

>>> def seperate():
        a = int(input('자연수 중 하나를 입력하세요: '))
        if a % 2 == 0:
            print('짝수')
        else:
            print('홀수')
```

부가세 계산

소비자 가격 = 물건 가격 × 1.1 · 물건 가격 = 소비자 가격 ÷ 1.1 · 부가세 = 물건 가격 × 0.1

03 나만의 텍스트 파일 가공하기

▶ os 모듈 — 현재 위치 설정하기

함수	설명
<code>import os</code>	os 모듈 импорт
<code>os.getcwd()</code>	현재 위치(작업 폴더) 확인
<code>os.chdir(경로)</code>	폴더 이동. 경로의 \ 는 \\ 로 쓰거나 <code>r'C:\Users\...'</code> 처럼 앞에 r을 붙인다
<code>os.listdir()</code>	폴더 안의 파일 목록을 리스트 로 반환

▶ 파일 열고 닫기

함수 / 메서드	형식	설명
<code>open()</code>	<code>파일 객체 = open('파일 이름', '모드')</code>	파일 열기
<code>close()</code>	<code>f.close()</code>	파일 닫기. 열었으면 꼭 닫아야 함
<code>write()</code>	<code>f.write('내용')</code>	파일에 쓰기. 입력한 문자 개수를 반환
<code>read()</code>	<code>f.read()</code> / <code>f.read(100)</code>	내용 읽기 (숫자를 넣으면 그 글자 수만큼)
<code>seek()</code>	<code>f.seek(0)</code>	커서를 파일 맨 앞으로 이동 (다시 읽을 때)
<code>readlines()</code>	<code>sentences = f.readlines()</code>	모든 줄을 원소로 하는 리스트 로 반환
<code>writelines()</code>	<code>newf.writelines(리스트)</code>	리스트의 원소를 한 줄씩 씀

▶ 파일 열기 모드

모드	의미
<code>'w'</code>	쓰기 — 파일에 내용을 새로 쓸 때 (기존 내용은 지워짐)
<code>'r'</code>	읽기 — 파일 내용을 읽을 때. 없는 파일이면 FileNotFoundError
<code>'a'</code>	추가 — 기존 내용 뒤에 이어서 쓸 때

```
>>> f = open('a.txt', 'r')
>>> diary = f.read()           # 읽은 내용을 객체에 저장하면 슬라이싱 등 문자열 기능 사용 가능
>>> print(diary[:5])
나는 오늘
>>> f.close()
```

with 문 — 자동으로 파일을 닫아 줌

```
with open(파일 이름, 모드) as f:
    f에 수행할 명령

>>> with open('test.txt', 'w') as f:
    f.write('오늘 나는 학교에 갔습니다.')
```

한글 파일 오류 해결 — encoding 지정

```
f = open('한글파일.txt', 'r', encoding='utf8')
f = open('monica.txt', 'w', encoding='utf8')
```

▶ 정규표현식 re 모듈 ★

특정한 문자의 규칙(패턴)을 찾고 가공하는 방법. 이메일·전화번호·인용구처럼 규칙이 있는 문자열을 찾을 때 유용. `import re` 로 불러온다.

메서드	형식	설명
<code>re.match()</code>	<code>re.match(패턴, 문자열)</code>	문자열 시작부터 패턴이 맞는지 확인. 중간에 있는 패턴은 못 찾음. 대·소문자 구분. 없으면 None
<code>re.search()</code>	<code>re.search(패턴, 문자열)</code>	문자열 전체 에서 패턴을 찾음 (중간에 있어도 찾음)
<code>.group()</code>	<code>re.match(...).group()</code>	찾은(일치한) 내용을 반환. 매칭 실패(None)면 AttributeError
<code>re.findall()</code>	<code>re.findall(패턴, 찾으려는 문자열)</code>	패턴을 모두 찾아 리스트 로 반환
<code>re.split()</code>	<code>re.split(패턴, 문자열)</code>	패턴이 나올 때마다 문자열을 나눠 리스트로 반환 (문장 나누기)
<code>re.sub()</code>	<code>re.sub(찾을 패턴, 대체할 문자, 찾을 문자열)</code>	패턴을 찾아 다른 문자로 바꿈 (불필요한 공백·줄바꿈 제거)
<code>re.compile()</code>	<code>char = re.compile(r'[A-Z][a-z]+')</code>	검색 조건(패턴)을 미리 컴파일해 객체로 저장

```
>>> re.match('life', 'life').group()
'life'
>>> re.search('cool', 'Life is so cool').group()
'cool'
>>> re.findall(r'\d{6}', 'My number is 511223-1***** and yours is 521012-2*****')
['511223', '521012']
>>> re.split(r'[.?!]', 'I have a lovely dog, really. I am not telling a lie. What a pretty dog! I love this dog.')
['I have a lovely dog, really.', 'I am not telling a lie.', 'What a pretty dog!', 'I love this dog.']
>>> re.sub('dog', 'cat', sentence)          # 'dog' → 'cat'
>>> re.sub(r'\n', '', words)                # 줄 바꿈 문자 제거
```

▶ 정규표현식 패턴 기호

기호	설명
<code>\d</code>	숫자와 매치. <code>[0-9]</code> 와 같음
<code>\D</code>	숫자가 아닌 것과 매치. <code>[^0-9]</code> 와 같음
<code>\s</code>	공백(whitespace) 문자와 매치. <code>[\t\n\r\f\v]</code>
<code>\S</code>	공백 문자가 아닌 것과 매치
<code>\w</code>	문자+숫자와 매치. <code>[a-zA-Z0-9_]</code>
<code>\W</code>	문자+숫자가 아닌 것과 매치. <code>[^a-zA-Z0-9_]</code>
<code>\\</code>	메타 문자가 아닌 일반 역슬래시(\). 메타 문자 앞에 \를 붙이면 일반 문자 (예: <code>\\(</code> , <code>\\.</code>)
<code>.</code> +	<code>.</code> = 아무 문자 하나, <code>+</code> = 앞 문자가 1번 이상 반복. <code>.+</code> = 아무 문자나 반복
<code>{n}</code>	앞 문자를 n번 반복. <code>\d{6}</code> = 숫자 6개
<code>[]</code>	괄호 안 문자 중 하나. <code>[A-Z]</code> 대문자, <code>[a-z]</code> 소문자, <code>[가-힣]</code> 한글
<code>r'...'</code>	정규식 문자열 앞에는 습관적으로 r을 붙인다 (역슬래시를 그대로 인식)

탐욕(greedy) 제어 — 마침표(.)와 물음표(?)

`.+` 는 가능한 한 길게(끝까지) 찾는다. 뒤에 `?` 를 붙이면 처음 만나는 지점에서 멈춘다.

`re.findall(r'\d.+년', example1)` → ['91년에 태어났습니다. 97년에는 IMF가 있었습니다. 지금은 2020년']

`re.findall(r'\d.+?년', example1)` → ['91년', '97년', '2020년'] (또는 `r'\d+년'`)

인용구: `re.findall(r'\((.+?)\)', example)` → ['이동민, 2019', '최재영, 2019', 'Lion, 2018']

▶ 문자열·집합 관련

함수 / 메서드	형식	설명
<code>replace()</code>	<code>문자열.replace('찾을 문자열', '바꿀 문자열')</code>	문자를 찾아 바꿈. <code>'22,333,422'.replace(',', '')</code> → '22333422'
<code>set()</code>	<code>set(리스트)</code>	중복값 제거 (집합). 다시 리스트로 쓰려면 <code>list()</code>
<code>list()</code>	<code>list(객체)</code>	리스트로 변환
슬라이싱	<code>문자열[시작:끝]</code>	<code>i[:-1]</code> → 마지막 글자(콜론 등) 제거
<code>+=</code>	<code>monica += i + '\n'</code>	문자열 이어 붙이기 (줄 바꿈 추가)

```
# 등장인물 이름만 중복 없이 한 줄로 모으기
>>> character = [x[:-1] for x in list(set(re.findall(r'[A-Z][a-z]+:', script101)))]
# would가 들어간 대사만 리스트로 모으기
>>> would = [i for i in sentences if re.match(r'[A-Z][a-z]+:', i) and re.search('would', i)]
```

04 CSV 파일로 실생활 데이터 다루기

CSV (Comma Separated Value)

쉼표로 나뉜 값들을 저장한 데이터. 엑셀에서 사용 가능하고, 서식 정보가 없어 원형 그대로 가공하기 좋다.

CSV형 리스트 = `[[1행], [2행], ..., [n행]]` — 리스트 안에 행 리스트가 들어 있는 형태.

CSV를 파이썬으로 불러오기 = **읽기(read)**, 가공한 내용을 CSV로 저장하기 = **쓰기(write)**

▶ csv 모듈 함수

함수 / 메서드	형식	설명
<code>import csv</code>	<code>import csv, os</code>	csv 모듈 임포트
<code>csv.reader()</code>	<code>new = csv.reader(f)</code>	열린 CSV 파일을 읽는 reader 객체 생성. for 문으로 한 행씩(리스트) 꺼냄
<code>append()</code>	<code>a_list.append(i)</code>	리스트에 원소 추가 → CSV형 리스트 만들기
<code>csv.writer()</code>	<code>csvobject = csv.writer(f, delimiter=',')</code>	CSV 파일에 쓰는 writer 객체 생성
<code>writerows()</code>	<code>csvobject.writerows(a)</code>	CSV형 리스트를 파일에 한꺼번에 저장

인코딩·newline

읽기: `open('a.csv', 'r', encoding='utf8')` · 쓰기:

`open('abc.csv', 'w', newline='', encoding='utf-8-sig')`

'utf-8-sig'로 지정하면 엑셀에서 한글이 깨지는 문제를 막을 수 있다. reader로 한 번 다 읽은 뒤 다시 읽으려면 `f.seek(0)`.

▶ 직접 만든 함수 — `opencsv()`, `writcsv()`

```
def opencsv(filename):
    f = open(filename, 'r')
    reader = csv.reader(f)
    output = []
    for i in reader:
        output.append(i)
    return output
# 사용: opencsv('example2.csv') → CSV형 리스트
```

```
def writcsv(filename, the_list):
    with open(filename, 'w', newline='',
              encoding='utf-8-sig') as f:
        a = csv.writer(f, delimiter=',')
        a.writerows(the_list)
# 사용: writcsv('test.csv', a)
```

▶ 나만의 모듈 — `usecsv`

`opencsv()`와 `writcsv()`(이후 `switch()`까지)를 **usecsv.py** 파일에 모아 모듈로 만든다. 파이썬 설치 경로의 **Lib 폴더**에 복사해 두면 어디서든 `import usecsv`로 사용 가능.

```
>>> import os, usecsv
>>> os.chdir(r'C:\Users\user\do-it-python\04')
>>> a = [['국어', '영어', '수학'], [99, 88, 77]]
>>> usecsv.writcsv('test.csv', a)           # 쓰기
>>> total = usecsv.opencsv('popSeoul2023.csv') # 읽기
```

▶ CSV 안의 문자를 숫자로 전환하기

CSV에서 읽은 값은 모두 문자열('152,212' 등). `replace()`로 쉼표를 지우고 `float()` / `int()`로 바꾼다.

```
>>> float('152,212'.replace(',', ''))
152212.0
```

작업 순서: ① 정규식 불러오기 → ② 숫자로 바꿀 수 있는 원소 찾기 → ③ 쉼표 지우기 → ④ int/float로 바꾸기 → ⑤ 숫자로 못 바꾸면 그대로 둔다

방법	핵심 코드
숫자 원소만 골라 변환	<code>if re.search(r'\d', j): k.append(float(re.sub(',', '', j)))</code>
문자·숫자 섞인 원소 제외	<code>if re.search('[a-zA-Z-]', j): k.append(j)</code> → 그대로 둠
제자리에서 바꾸기	<code>i[i.index(j)] = float(j.replace(',', ''))</code> — <code>index()</code> 로 원소 위치를 찾아 교체
예외 처리로 넘어가기	<code>try:</code> 변환 <code>except: pass</code> — 바꿀 수 없는 원소는 건너뛰

▶ usecsv에 추가한 switch() 함수

```
def switch(listName):
    for i in listName:
        for j in i:
            try:
                i[i.index(j)] = float(j.replace(',', ''))
            except:
                pass
    return listName
# 사용: newPop = usecsv.switch(total) → 숫자로 바꿀 수 있는 원소만 실수형으로
```

▶ 분석 활용 패턴

```
# 외국인 비율 3% 초과 구만 골라 새 CSV로 저장
new = [['구', '한국인', '외국인', '외국인 비율(%)']] # 첫 행 지정
for i in newPop:
    try:
        foreign = round(i[3] / (i[2] + i[3]) * 100, 1)
        if foreign > 3:
            new.append([i[0], i[2], i[3], foreign])
    except:
        pass
usecsv.writecsv('newPop2023.csv', new)

# 조건 검색: 강원 지역, 120㎡ 이상, 3억 원 이하 아파트
apt = usecsv.switch(usecsv.opencsv('apt2308.csv'))
if i[5] >= 120 and i[8] <= 30000 and re.match('강원', i[0]):

# 영어·한국어 문장을 나눠 표로 저장
Korean_list = re.split(r'\.', Korean)
English_list = re.split(r'\.', English)
for i in range(len(English_list)):
    total.append([English_list[i], Korean_list[i]])
usecsv.writecsv('Korean_English.csv', total)
```

함수	설명
<code>len()</code>	원소 개수. <code>len(apt)</code> → 거래 건수
<code>index()</code>	리스트에서 원소의 위치(인덱스) 반환
<code>range(len(리스트))</code>	리스트 길이만큼 0부터 반복 (두 리스트를 같은 위치로 묶을 때)